

Assignment 1B: Dynamic Programming

Carlo Dubbioso (s4416090) & Maria Mukhovykova (s4379055)

Group Number: 55

1 Introduction

Dynamic Programming (DP) is a type of algorithm that computes optimal decisions in environments that can be built as Markov Decision Processes (MDPs). DP acts as a bridge between planning and learning through experience. Compared to reinforcement learning, which samples interactions from the environment, DP takes full knowledge of the environment using transition probabilities $p(s'|s, a)$ and expected reward $r(s, a, s')$.

In this assignment, we use DP to solve MDPs and get the optimal policies. The environment that we studied is Windy GridWorld which consists of 7x6 grid, where at each cell we can produce four moves: up, down, left, and right. Additionally, there three columns with upward “wind” that moves the agent vertically by one or two cells.

The goal is to understand how DP methods use the mathematical framework of the Bellman equation to derive optimal value functions and policies. For that, we implement Policy Iteration Agent and Value Iteration Agent.

2 Markov Decision Process (MDP)

To explain Dynamic Programming methods, we use Markov Decision Process (MDP) for modeling Windy GridWorld. We define MDP as:

- States (S) – all situations in which agent can be
- Actions (A) – a set of actions that the agent can perform
- Transition probabilities $P(s'|s, a)$ - the probability of moving to state s' by taking action a in state s
- Reward function $R(s, a, s')$ - reward that is received after transition
- Discounted factor $\gamma \in [0, 1]$ - how future rewards are weighted compared to immediate.

The discount factor γ has a very important role in agent's objective. When $\gamma = 1$ experiments as reaching the goal in the grid world. However, when $\gamma < 1$, there are several reasons to use it: it ensures the sum of future rewards converges, it prefers shorter paths by downweighing further rewards, and it makes the agent focus on closer gains instead of far-off rewards. Moreover, when the environment doesn't have a step penalty (reward_per_step = 0), using $\gamma < 1$ ensures that the agent gets to the goal faster and doesn't enter infinite loop.

3 Dynamic Programming

Dynamic Programming is a group of algorithms that are used to compute optimal policies by applying Bellman equations. DP algorithms compute value functions that estimate expected return from the provided state or state-action pair. These values get updated until they converge to stable estimates and after that the policy can be deduced by taking the action with the highest value.

There are two common algorithms in DP: Policy Iteration and Value Iteration

3.1 Policy Iteration

Policy Iteration moves between two phases. First step is policy evaluation, where we apply algorithm to compute value function for the given policy. This is achieved by using Bellman update. Second step is policy improvement where we choose the action that maximizes expected value. This repeats until the policy doesn't change anymore which guarantees the optimal policy.

3.2 Value Iteration

With value iteration we combine those two phases in a single update step. This algorithm immediately updates value estimates by choosing the maximum one over actions.

4 Generalized Policy Iteration

Generalized Policy Iteration (GPI) is the combination of value estimation and policy improvement. GPI shows the main principles behind reinforcement learning and planning algorithms.

In GPI two phases happen at the same time: Policy Evaluation and Policy Improvement. These steps influence each other, as value estimate gets more accurate, policy improves, and when policy improves, estimates adjust.

5 The Bellman Equation

The Bellman Equation is used for value updates of state-action pairs.

$$Q^\pi(s, a) = \sum_{s', r} p(s', r|s, a)[r + \gamma \sum_{a'} \pi(a', s') Q^\pi(s', a')]$$

The components:

- $Q^\pi(s, a)$ – the expected return when taking action a in state s with the policy π
- $p(s', r|s, a)$ – a probability of transitioning to state s' with reward r
- r - immediate reward
- γ - discount factor
- $\pi(a', s')$ - the probability of choosing action a' in state s'

The key idea of this equation is that the value of state-action pair equals to the immediate reward plus discounted expected value of future states [1].

6 Results

Figure 1 shows the Windy GridWorld environment with calculated Q -values and optimal policy. As the step reward is set to 0 and the discount factor to 1, most of the states have equal expected return - 35, which is the same as the final reward. The arrows in each state represent the optimal actions that show where the agent should move to reach the goal.

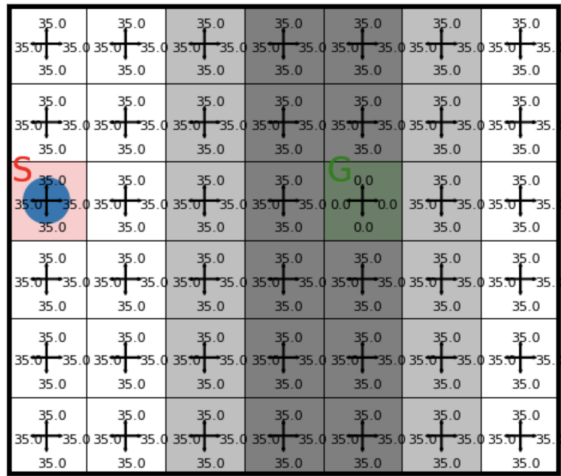


Figure 1: Optimal policy and Q-values for the deterministic Windy GridWorld environment using Value Iteration.

In Figure 2 we can observe Windy GridWorld environment with stochastic wind, where the wind occurs only with probability 0.75. Compared to Figure 1, which has deterministic wind, this environment shows randomness in the wind strength. As the agent should consider the wind strength to choose the best action, the optimal policy can be different from the deterministic case. However, because the Bellman update includes transition probabilities, the value function can still represent the expected results of different possible transitions.

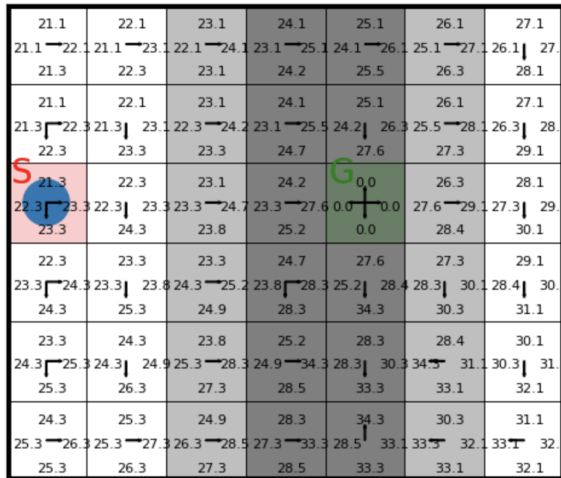


Figure 2: Optimal policy and Q-values for Windy GridWorld with stochastic wind (0.75).

7 Conclusion

Dynamic Programming is a good tool to solve Markov Decision Processes with known environment model. The algorithms Policy Iteration and Value Iteration compute optimal policies by repeatedly

using Bellman equation. These methods are practical implementation of Generalized Policy Iteration, two phases - policy evaluation and policy improvement - interact until convergence.

In the Windy GridWorld environment we apply DP methods to see how they learn optimal strategies even in stochastic case. Dynamic Programming considered to be an important theoretical basis for modern reinforcement learning algorithms, even though it has its own limitations.

References

- [1] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2018.